

The Chinese Remainder Theorem

A Lecture: Statement, Proofs, Algorithms, and Applications

Contents

1	Motivation	1
2	The Two-Modulus Case	1
2.1	Proof 1: Explicit construction via Bézout	2
2.2	Proof 2: Pigeonhole / counting argument	3
3	The General Case: k Pairwise Coprime Moduli	3
4	The Algebraic Formulation: A Ring Isomorphism	4
5	CRT with Non-Coprime Moduli	5
6	The Polynomial Chinese Remainder Theorem	6
7	Applications	7
7.1	Multiplicativity of Euler's totient function	7
7.2	Fast RSA decryption: Garner's algorithm / CRT-RSA	7
7.3	Secret sharing (Asmuth–Bloom scheme)	8
7.4	Computer arithmetic: Residue Number Systems (RNS)	8
7.5	Fast algorithms: multiplying big numbers, FFT-based convolution	8
7.6	Error-correcting codes	8
8	Summary of Key Formulas	9
9	Exercises	9

1 Motivation

The Chinese Remainder Theorem (CRT) answers a question that sounds almost too simple to be deep: *if I tell you the remainder of a number when divided by several different, pairwise coprime numbers, can you tell me the number itself (modulo their product)?* The answer is yes, uniquely, and the reconstruction is completely explicit. This idea dates back at least to Sunzi's mathematical manual (China, 3rd–5th century CE), which posed a version of the riddle:

There are certain things whose number is unknown. Repeatedly divided by 3, the remainder is 2; by 5, the remainder is 3; by 7, the remainder is 2. What is the number?

We will answer this exact riddle later (Example 3.3). The theorem turns out to be one of the load-bearing pillars of number theory and abstract algebra: it explains why φ is multiplicative, it underlies fast modular arithmetic in cryptography, it powers secret-sharing schemes, and — in its most general algebraic form — it is a statement about how a ring decomposes into a product of simpler rings.

In plain English: think of a number x as being “coded” by its remainders modulo several small numbers. CRT says that if those small numbers share no common factors pairwise, then the code is lossless and invertible: the collection of remainders determines x completely (mod the product), and there’s an explicit recipe to decode it.

2 The Two-Modulus Case

Theorem 2.1 (CRT, two moduli). *Let m, n be positive integers with $\gcd(m, n) = 1$. Then for any integers a, b , the system*

$$x \equiv a \pmod{m}, \quad x \equiv b \pmod{n}$$

has a solution $x \in \mathbb{Z}$, and this solution is unique modulo mn . That is, there exists x_0 with $0 \leq x_0 < mn$ solving the system, and the full solution set is $\{x_0 + kmn : k \in \mathbb{Z}\}$.

We prove this in two independent ways: first by an explicit construction using Bézout’s identity (this is the version you would program on a computer), and second by a slicker counting/pigeonhole argument (this is the version that generalizes best conceptually).

2.1 Proof 1: Explicit construction via Bézout

Lemma 2.2 (Bézout’s identity). *If $\gcd(m, n) = 1$, there exist integers u, v such that*

$$mu + nv = 1.$$

Proof. This is a standard consequence of the Euclidean algorithm: running the algorithm on (m, n) to compute $\gcd(m, n)$ and then back-substituting each division step expresses the final gcd, 1, as an integer combination $mu + nv$. (See any elementary number theory text for the full back-substitution; we take this as known machinery.) $\square$

Proof of Theorem 2.1, construction. By Lemma 2.2, choose integers u, v with

$$mu + nv = 1.$$

Define

$$x_0 = a nv + b mu.$$

Check the first congruence. Since $mu = 1 - nv$, we have $b mu = b(1 - nv) = b - bnv$, so

$$x_0 = anv + b - bnv = b + (a - b)nv.$$

Hmm — let’s instead check directly and cleanly. Modulo m : since $mu \equiv 0 \pmod{m}$, we get $nv \equiv 1 - mu \equiv 1 \pmod{m}$. Therefore

$$x_0 = a(nv) + b(mu) \equiv a \cdot 1 + b \cdot 0 = a \pmod{m}.$$

Check the second congruence. Modulo n : since $nv \equiv 0 \pmod{n}$, we get $mu \equiv 1 - nv \equiv 1 \pmod{n}$. Therefore

$$x_0 = a(nv) + b(mu) \equiv a \cdot 0 + b \cdot 1 = b \pmod{n}.$$

So x_0 solves both congruences simultaneously. This proves **existence**.

Uniqueness modulo mn . Suppose x_0 and x_1 both solve the system. Then

$$x_0 \equiv x_1 \pmod{m} \quad \text{and} \quad x_0 \equiv x_1 \pmod{n},$$

i.e. $m \mid (x_0 - x_1)$ and $n \mid (x_0 - x_1)$. Since $\gcd(m, n) = 1$, this forces $mn \mid (x_0 - x_1)$ (a standard fact: if two coprime numbers each divide D , their product divides D — this itself follows from unique factorization, since m and n share no prime factors, so all prime-power factors of m and of n must appear in D simultaneously). Hence $x_0 \equiv x_1 \pmod{mn}$. $\square$

In plain English: Bézout’s identity gives us two special “indicator” numbers: $e_1 = nv$ is $\equiv 1 \pmod{m}$ but $\equiv 0 \pmod{n}$, and $e_2 = mu$ is $\equiv 0 \pmod{m}$ but $\equiv 1 \pmod{n}$. These act like a basis: to hit target remainder $a \pmod{m}$ and $b \pmod{n}$ simultaneously, just take the combination $a \cdot e_1 + b \cdot e_2$ — each term contributes exactly where it’s supposed to and vanishes where it isn’t supposed to interfere. This “indicator function” idea is exactly the same trick used in Lagrange interpolation (see Section 6).

2.2 Proof 2: Pigeonhole / counting argument

Proof of Theorem 2.1, counting. Consider the function

$$f : \{0, 1, \dots, mn - 1\} \longrightarrow \{0, \dots, m - 1\} \times \{0, \dots, n - 1\}, \quad f(x) = (x \bmod m, x \bmod n).$$

Both sets have exactly mn elements, so f is a bijection if and only if it is injective. Suppose $f(x) = f(y)$ for $x, y \in \{0, \dots, mn - 1\}$. Then $x \equiv y \pmod{m}$ and $x \equiv y \pmod{n}$, so as in the uniqueness argument above, $mn \mid (x - y)$. But $|x - y| < mn$, so $x = y$. Hence f is injective, and since the domain and codomain are finite of equal size, f is a bijection.

In particular, every pair $(a \bmod m, b \bmod n)$ is hit by exactly one $x \in \{0, \dots, mn - 1\}$ — this x is simultaneously the existence and the uniqueness statement of the theorem. $\square$

In plain English: there are exactly mn numbers from 0 to $mn - 1$, and exactly mn possible pairs of remainders $(\bmod m, \bmod n)$. If two different numbers in that range could never collide (produce the same pair of remainders) — which is what coprimality guarantees — then, since the counts match exactly, every possible pair of remainders *must* be achieved by some number, and only one.

Example 2.3. Solve $x \equiv 2 \pmod{3}$, $x \equiv 3 \pmod{5}$. Here $m = 3, n = 5$. Bézout: $3(2) + 5(-1) = 1$, so $u = 2, v = -1$. Then $nv = 5(-1) = -5 \equiv 1 \pmod{3}$, $\equiv 0 \pmod{5}$, and $mu = 3(2) = 6 \equiv 0 \pmod{3}$, $\equiv 1 \pmod{5}$. So

$$x_0 = a \cdot nv + b \cdot mu = 2(-5) + 3(6) = -10 + 18 = 8.$$

Check: $8 = 2 \cdot 3 + 2$, remainder 2 $\pmod{3}$ $\checkmark$; $8 = 1 \cdot 5 + 3$, remainder 3 $\pmod{5}$ $\checkmark$. The full solution set is $x \equiv 8 \pmod{15}$.

3 The General Case: k Pairwise Coprime Moduli

Theorem 3.1 (CRT, general form). *Let $m_1, m_2, \dots, m_k$ be pairwise coprime positive integers (i.e. $\gcd(m_i, m_j) = 1$ for $i \neq j$), and let $M = m_1 m_2 \cdots m_k$. For any integers $a_1, \dots, a_k$, the system*

$$x \equiv a_1 \pmod{m_1}, \quad x \equiv a_2 \pmod{m_2}, \quad \dots, \quad x \equiv a_k \pmod{m_k}$$

has a solution, unique modulo M .

Proof. Induction on k . The base case $k = 1$ is trivial ($x \equiv a_1 \pmod{m_1}$ is already the full statement), and $k = 2$ is Theorem 2.1.

Inductive step. Suppose the theorem holds for $k - 1$ moduli. Given pairwise coprime $m_1, \dots, m_k$, first solve the subsystem in the first $k - 1$ congruences: by the inductive hypothesis, there is a unique y_0 modulo $M' := m_1 \cdots m_{k-1}$ satisfying

$$x \equiv a_i \pmod{m_i}, \quad i = 1, \dots, k - 1.$$

It remains to solve simultaneously

$$x \equiv y_0 \pmod{M'}, \quad x \equiv a_k \pmod{m_k}.$$

Key point: $\gcd(M', m_k) = 1$. Indeed, $M' = m_1 \cdots m_{k-1}$, and m_k is coprime to each m_i ($i < k$) by hypothesis; a number coprime to each of several integers is coprime to their product (no prime factor of m_k can divide M' , since such a prime would have to divide some m_i , contradicting $\gcd(m_i, m_k) = 1$). So $\gcd(M', m_k) = 1$, and Theorem 2.1 (two-modulus case) applies directly to this pair, giving existence and uniqueness of x modulo $M'm_k = M$.

This completes the induction. $\square$

In plain English: you don't need a fundamentally new idea for many moduli — you just fold them in one at a time. Solve the first two, combine them into a single congruence modulo their product, then combine that with the third modulus (which is still coprime to the product, since it was coprime to each factor separately), and so on. This is also exactly how you would implement CRT reconstruction as an algorithm.

Algorithm 3.2 (Iterative CRT reconstruction). Given pairwise coprime $m_1, \dots, m_k$ and targets $a_1, \dots, a_k$:

1. Initialize $x \leftarrow a_1$, $M \leftarrow m_1$.
2. For $i = 2, \dots, k$: find x' satisfying $x' \equiv x \pmod{M}$ and $x' \equiv a_i \pmod{m_i}$, using the two-modulus formula of Theorem 2.1 (Bézout construction) on the coprime pair (M, m_i) . Update $x \leftarrow x' \pmod{Mm_i}$, $M \leftarrow Mm_i$.
3. Return x (the unique solution modulo $M = m_1 \cdots m_k$).

Example 3.3 (Sunzi's Problem). Solve $x \equiv 2 \pmod{3}$, $x \equiv 3 \pmod{5}$, $x \equiv 2 \pmod{7}$.

Step 1. From Example 2.3, $x \equiv 2 \pmod{3}$ and $x \equiv 3 \pmod{5}$ combine to $x \equiv 8 \pmod{15}$.

Step 2. Combine $x \equiv 8 \pmod{15}$ with $x \equiv 2 \pmod{7}$. Bézout for $(15, 7)$: $15(1) + 7(-2) = 1$, so $u = 1$, $v = -2$ (here " m " = 15, " n " = 7). Then $nv = 7(-2) = -14 \equiv 1 \pmod{15}$, $\equiv 0 \pmod{7}$, and $mu = 15(1) = 15 \equiv 0 \pmod{15}$, $\equiv 1 \pmod{7}$. So

$$x_0 = 8 \cdot (-14) + 2 \cdot (15) = -112 + 30 = -82 \equiv -82 + 105 = 23 \pmod{105}.$$

Check: $23 = 7 \cdot 3 + 2 \pmod{3} \checkmark$, $23 = 4 \cdot 5 + 3 \pmod{5} \checkmark$, $23 = 3 \cdot 7 + 2 \pmod{7} \checkmark$.

So the smallest positive answer to Sunzi's ancient riddle is $\boxed{x = 23}$, with general solution $x \equiv 23 \pmod{105}$.

Example 3.4. Solve $x \equiv 1 \pmod{4}$, $x \equiv 2 \pmod{9}$, $x \equiv 3 \pmod{25}$ (moduli pairwise coprime, product = 900). Combine first two: need $x \equiv 1 \pmod{4}$, $x \equiv 2 \pmod{9}$. Try $x = 29$: $29 = 7 \cdot 4 + 1 \checkmark$, $29 = 3 \cdot 9 + 2 \checkmark$. So $x \equiv 29 \pmod{36}$. Now combine with $x \equiv 3 \pmod{25}$: Bézout for $(36, 25)$: $36(-4) + 25(6) = -144 + 150 = 6 \dots$ adjust: try $36(4) + 25(-\dots)$. Use extended Euclid:

$36 = 1 \cdot 25 + 11$; $25 = 2 \cdot 11 + 3$; $11 = 3 \cdot 3 + 2$; $3 = 1 \cdot 2 + 1$. Back-substitute: $1 = 3 - 1 \cdot 2 = 3 - 1(11 - 3 \cdot 3) = 4 \cdot 3 - 11 = 4(25 - 2 \cdot 11) - 11 = 4 \cdot 25 - 9 \cdot 11 = 4 \cdot 25 - 9(36 - 25) = 13 \cdot 25 - 9 \cdot 36$. So $36(-9) + 25(13) = 1$, i.e. $u = -9, v = 13$ for $(m, n) = (36, 25)$. Then $nv = 25 \cdot 13 = 325 \equiv 1 \pmod{36}$ (since $325 = 9 \cdot 36 + 1$), $\equiv 0 \pmod{25}$; $mu = 36(-9) = -324 \equiv 0 \pmod{36}$, $\equiv 1 \pmod{25}$ (since $-324 = -13 \cdot 25 + 1$).

$$x_0 = 29(325) + 3(-324) = 9425 - 972 = 8453 \equiv 8453 - 9 \cdot 900 = 8453 - 8100 = 353 \pmod{900}.$$

Check: $353 = 88 \cdot 4 + 1\checkmark$; $353 = 39 \cdot 9 + 2\checkmark$; $353 = 14 \cdot 25 + 3\checkmark$. So $x \equiv 353 \pmod{900}$.

4 The Algebraic Formulation: A Ring Isomorphism

The deepest and most reusable version of CRT is not a statement about solving congruences, but a statement about the *structure* of the ring $\mathbb{Z}/M\mathbb{Z}$ itself.

Theorem 4.1 (CRT as ring isomorphism). *Let $m_1, \dots, m_k$ be pairwise coprime, $M = m_1 \cdots m_k$. The map*

$$\Psi : \mathbb{Z}/M\mathbb{Z} \longrightarrow \mathbb{Z}/m_1\mathbb{Z} \times \mathbb{Z}/m_2\mathbb{Z} \times \cdots \times \mathbb{Z}/m_k\mathbb{Z}, \quad \Psi(x \bmod M) = (x \bmod m_1, \dots, x \bmod m_k)$$

is a ring isomorphism: it is a bijection that respects both addition and multiplication,

$$\Psi(x + y) = \Psi(x) + \Psi(y), \quad \Psi(xy) = \Psi(x)\Psi(y),$$

where addition and multiplication on the right side are done componentwise.

Proof. Well-defined: if $x \equiv x' \pmod{M}$ then $x \equiv x' \pmod{m_i}$ for each i (since $m_i \mid M$), so Ψ does not depend on the choice of representative x .

Bijjective: this is exactly Theorem 3.1 — every tuple $(a_1, \dots, a_k)$ arises from a unique $x \bmod M$, so Ψ is a bijection.

Respects addition: $(x + y) \bmod m_i = ((x \bmod m_i) + (y \bmod m_i)) \bmod m_i$ for each i — this is just the elementary fact that reduction mod m_i is compatible with addition. Hence $\Psi(x + y) = \Psi(x) + \Psi(y)$ componentwise.

Respects multiplication: identically, $(xy) \bmod m_i = ((x \bmod m_i)(y \bmod m_i)) \bmod m_i$ for each i , so $\Psi(xy) = \Psi(x)\Psi(y)$.

A bijection compatible with both ring operations is by definition a ring isomorphism. $\square$

In plain English: arithmetic mod M is *secretly the same thing* as doing arithmetic mod m_1 , mod m_2 , $\dots$, mod m_k all simultaneously and independently, then reassembling the answer. You can add, subtract, or multiply numbers coordinate-by-coordinate in the separate small moduli, and it always agrees with doing the operation directly in the big modulus M . This is not just a coincidence about congruences; it is a full structural equivalence of algebraic systems (rings).

Corollary 4.2 (Units transfer). *Under Ψ , an element $x \bmod M$ is a unit (invertible under multiplication) in $\mathbb{Z}/M\mathbb{Z}$ if and only if each component $x \bmod m_i$ is a unit in $\mathbb{Z}/m_i\mathbb{Z}$. Consequently,*

$$(\mathbb{Z}/M\mathbb{Z})^\times \cong (\mathbb{Z}/m_1\mathbb{Z})^\times \times \cdots \times (\mathbb{Z}/m_k\mathbb{Z})^\times$$

as groups (in fact, as multiplicative monoids restricted to units), and taking cardinalities recovers

$$\varphi(M) = \varphi(m_1)\varphi(m_2)\cdots\varphi(m_k),$$

i.e. the multiplicativity of Euler's totient function is a direct corollary of the CRT ring isomorphism.

Proof. A ring isomorphism sends units to units and non-units to non-units (an inverse in one ring corresponds, under the isomorphism, to an inverse in the other — apply Ψ to the equation $xy = 1$). Units in a product ring $R_1 \times \cdots \times R_k$ are exactly tuples that are units in *every* coordinate (invert coordinatewise). Restricting the bijection Ψ to unit groups gives the stated group isomorphism, and cardinalities multiply because the cardinality of a Cartesian product is the product of cardinalities. $\square$

This is precisely the argument used (in more computational language) in the earlier lecture on Euler's totient function — CRT is the machine underneath multiplicativity of φ .

5 CRT with Non-Coprime Moduli

What if the moduli are *not* pairwise coprime? The system may or may not have a solution, and when it does, the modulus of the combined solution shrinks from the product to the least common multiple.

Theorem 5.1 (Generalized CRT). *Let m, n be positive integers (not necessarily coprime) and $a, b \in \mathbb{Z}$. The system*

$$x \equiv a \pmod{m}, \quad x \equiv b \pmod{n}$$

has a solution if and only if

$$a \equiv b \pmod{\gcd(m, n)}.$$

When a solution exists, it is unique modulo $\text{lcm}(m, n)$.

Proof. Let $g = \gcd(m, n)$, $M = \text{lcm}(m, n)$.

Necessity. If x solves both congruences, then $x = a + sm = b + tn$ for integers s, t , so $a - b = tn - sm$. Since $g \mid m$ and $g \mid n$, the right side is divisible by g , so $g \mid (a - b)$, i.e. $a \equiv b \pmod{g}$.

Sufficiency and construction. Suppose $a \equiv b \pmod{g}$, i.e. $g \mid (b - a)$. By Bézout, there exist u, v with $mu + nv = g$. Then

$$b - a = \left(\frac{b - a}{g}\right)g = \left(\frac{b - a}{g}\right)(mu + nv).$$

Set $x_0 = a + mu \cdot \frac{b - a}{g}$. Then $x_0 \equiv a \pmod{m}$ trivially (we added a multiple of m). Also

$$x_0 = a + \frac{(b - a)}{g}(g - nv) = a + (b - a) - \frac{(b - a)}{g}nv = b - \frac{(b - a)}{g}nv \equiv b \pmod{n},$$

since we subtracted a multiple of n . So x_0 solves the system.

Uniqueness modulo $M = \text{lcm}(m, n)$. If x_0, x_1 both solve the system then $m \mid (x_0 - x_1)$ and $n \mid (x_0 - x_1)$, so $\text{lcm}(m, n) \mid (x_0 - x_1)$ (the lcm is, by definition, the smallest — and in fact the unique minimal generator of the set of — common multiples; any common multiple of m, n is a multiple of $\text{lcm}(m, n)$, a standard fact from the prime-factorization description of lcm and gcd). Hence $x_0 \equiv x_1 \pmod{M}$. $\square$

In plain English: if m, n share a common factor g , then knowing $x \bmod m$ already pins down $x \bmod g$, and knowing $x \bmod n$ also pins down $x \bmod g$ — so for the system to be consistent at all, these two implied values mod g must agree. If they do agree, there's no further obstruction, and

the combined information determines x up to the least common multiple (the smallest modulus consistent with both individual congruences) rather than the full product mn (which would double-count the shared factor g).

Example 5.2. Solve $x \equiv 4 \pmod{6}$, $x \equiv 1 \pmod{9}$. Here $g = \gcd(6, 9) = 3$. Check consistency: $4 \bmod 3 = 1$, $1 \bmod 3 = 1$ — they agree, so a solution exists. $M = \text{lcm}(6, 9) = 18$. Try $x = 10$: $10 = 1 \cdot 6 + 4\checkmark$, $10 = 1 \cdot 9 + 1\checkmark$. So $x \equiv 10 \pmod{18}$.

By contrast, $x \equiv 2 \pmod{4}$, $x \equiv 1 \pmod{6}$ has *no* solution: $g = \gcd(4, 6) = 2$, and $2 \bmod 2 = 0$ while $1 \bmod 2 = 1$ — inconsistent.

6 The Polynomial Chinese Remainder Theorem

CRT is not special to $\mathbb{Z}$; it holds in any **principal ideal domain**, and the polynomial ring $F[x]$ over a field F is a particularly important example, because it directly explains **Lagrange interpolation**.

Theorem 6.1 (CRT for polynomials). *Let F be a field and $p_1(x), \dots, p_k(x) \in F[x]$ be pairwise coprime polynomials (no common roots/factors), with $P(x) = p_1(x) \cdots p_k(x)$. For any polynomials $a_1(x), \dots, a_k(x)$, there is a polynomial $g(x)$, unique modulo $P(x)$, satisfying*

$$g(x) \equiv a_i(x) \pmod{p_i(x)}, \quad i = 1, \dots, k.$$

Proof sketch. Identical in structure to Theorem 3.1: because $F[x]$ has a Euclidean algorithm (polynomial long division), Bézout's identity holds for coprime polynomials, $p_i(x)u(x) + p_j(x)v(x) = 1$, and the same construction and induction go through verbatim, replacing integers with polynomials throughout. $\square$

Corollary 6.2 (Lagrange Interpolation as a special case). *Take $p_i(x) = x - c_i$ for k distinct points $c_1, \dots, c_k \in F$ (these are pairwise coprime linear polynomials since they have distinct roots), and let $a_i(x) = y_i$ (constants) be target values. The unique polynomial $g(x)$ of degree $< k$ with $g(c_i) = y_i$ for all i (interpolation through the k points (c_i, y_i)) is precisely the CRT reconstruction, and it recovers the classical Lagrange interpolation formula*

$$g(x) = \sum_{i=1}^k y_i \prod_{j \neq i} \frac{x - c_j}{c_i - c_j}.$$

In plain English: “knowing the remainder of $g(x)$ modulo $(x - c_i)$ ” is exactly the same as “knowing the value $g(c_i)$ ” (by the polynomial remainder theorem). So reconstructing a polynomial from its values at several points (interpolation) and reconstructing an integer from its remainders modulo several numbers (CRT) are, structurally, *the exact same theorem*, just applied in two different rings ($F[x]$ versus $\mathbb{Z}$). This is a beautiful example of how abstract algebra unifies superficially unrelated classical results.

7 Applications

7.1 Multiplicativity of Euler's totient function

As shown in Corollary 4.2, $\varphi(mn) = \varphi(m)\varphi(n)$ for coprime m, n is an immediate consequence of the CRT ring isomorphism restricted to unit groups. This is the same fact used (with an equivalent but more computational proof) in the companion lecture on the Euler totient function.

7.2 Fast RSA decryption: Garner’s algorithm / CRT-RSA

In RSA with modulus $n = pq$, a private-key holder who knows p, q separately can decrypt *roughly four times faster* using CRT:

- Compute $m_p = c^{d \bmod (p-1)} \bmod p$ and $m_q = c^{d \bmod (q-1)} \bmod q$ — two *much smaller* modular exponentiations (roughly half the bit-length each, and exponentiation cost grows worse than linearly in bit-length).
- Recombine m_p, m_q into $m \bmod n$ using the CRT formula of Theorem 2.1 (a fixed procedure known as **Garner’s algorithm**, which precomputes the Bézout-type coefficients once per key).

This CRT-based speedup is used in essentially every real-world RSA implementation.

7.3 Secret sharing (Asmuth–Bloom scheme)

CRT-based secret sharing splits a secret S into k shares such that any t of them can reconstruct S , but fewer cannot. Roughly: choose pairwise coprime moduli $m_1 < m_2 < \dots < m_k$ such that the product of any t of them exceeds S , but the product of any $t - 1$ of them does not. The secret is encoded as $S \bmod (m_1 \cdots m_t)$ -type residues, distributed as shares $S \bmod m_i$; any t shares let you reconstruct $S \pmod{\text{the product of those } t \text{ moduli}}$, which is large enough to pin S down exactly via CRT, while $t - 1$ shares leave genuine ambiguity.

7.4 Computer arithmetic: Residue Number Systems (RNS)

Large-integer arithmetic (as used in cryptography and signal processing) can represent a big number x as a tuple of remainders $(x \bmod m_1, \dots, x \bmod m_k)$ for pairwise coprime word-sized m_i . Addition, subtraction, and multiplication of the big number can then be done *independently and in parallel* in each small modulus (Theorem 4.1), which is far faster on hardware than doing the arithmetic directly on the large number, at the cost of needing CRT reconstruction (or avoiding it entirely, if only sums/products of residues are needed later) when a normal decimal/binary representation is required.

7.5 Fast algorithms: multiplying big numbers, FFT-based convolution

Some fast multiplication algorithms (and computations of convolutions, relevant to signal processing and polynomial multiplication) use CRT — either in $\mathbb{Z}$ over several prime moduli (to avoid overflow, then reconstructing at the end) or via the polynomial CRT of Theorem 6.1 (splitting a polynomial multiplication problem into evaluation at several points, multiplying pointwise, then interpolating back — this is precisely the philosophy behind the Fast Fourier Transform’s evaluate–multiply–interpolate strategy, where the “points” are roots of unity).

7.6 Error-correcting codes

Certain codes (e.g., Redundant Residue Number System codes) encode data redundantly using more moduli than strictly necessary for reconstruction; if a small number of residues are corrupted, CRT-like reconstruction from the remaining consistent residues can detect and correct the error, analogous to redundancy-based error correction in other coding schemes.

8 Summary of Key Formulas

Statement	Formula / Content
Two-modulus CRT	$x \equiv a \pmod{m}, x \equiv b \pmod{n}, \gcd(m, n) = 1 \Rightarrow \text{unique } x \pmod{mn}$
Bézout construction	$mu + nv = 1 \Rightarrow x_0 = a nv + b mu$
General CRT	pairwise coprime $m_1, \dots, m_k \Rightarrow \text{unique } x \pmod{M = \prod m_i}$
Ring isomorphism	$\mathbb{Z}/M\mathbb{Z} \cong \mathbb{Z}/m_1\mathbb{Z} \times \dots \times \mathbb{Z}/m_k\mathbb{Z}$
Units transfer	$(\mathbb{Z}/M\mathbb{Z})^\times \cong \prod_i (\mathbb{Z}/m_i\mathbb{Z})^\times$
Totient corollary	$\varphi(M) = \prod_i \varphi(m_i)$
Non-coprime case	solvable $\iff a \equiv b \pmod{\gcd(m, n)}$; unique $\pmod{\text{lcm}(m, n)}$
Polynomial CRT	pairwise coprime $p_i(x) \Rightarrow \text{unique } g(x) \pmod{\prod p_i(x)}$
Lagrange interpolation	special case with $p_i(x) = x - c_i$

9 Exercises

- (1) Solve $x \equiv 3 \pmod{7}$, $x \equiv 5 \pmod{11}$, $x \equiv 7 \pmod{13}$ using the iterative algorithm (Algorithm 3.2).
- (2) Determine whether $x \equiv 5 \pmod{12}$, $x \equiv 1 \pmod{18}$ has a solution; if so, find all solutions.
- (3) Prove that if $m_1, \dots, m_k$ are pairwise coprime and each $m_i \geq 2$, then the number of solutions to $x^2 \equiv 1 \pmod{M}$ (with $M = \prod m_i$) equals 2^k when every m_i is odd (relate this to the unit-group isomorphism of Corollary 4.2 and the number of square roots of 1 in each cyclic-or-not factor).
- (4) Using the polynomial CRT, reconstruct the unique polynomial of degree < 3 with $g(0) = 1$, $g(1) = 2$, $g(2) = 5$, and verify it matches the Lagrange interpolation formula.
- (5) Explain, in your own words, why CRT-RSA decryption is faster, using the fact that modular exponentiation cost grows more than linearly in the bit-length of the modulus.
- (6) Show that $\mathbb{Z}/6\mathbb{Z} \cong \mathbb{Z}/2\mathbb{Z} \times \mathbb{Z}/3\mathbb{Z}$ explicitly, by writing out the full addition and multiplication tables on both sides and matching them via Ψ .